Міністерство освіти і науки України

Національний технічний університет України «КПІ імені Ігоря Сікорського»

Кафедра Інформаційних Систем та Технологій

Звіт

Лабораторної роботи №3

з дисципліни **«Розробка мобільних застосувань під Android»**

на тему **«Дослідження способів збереження даних»**

Перевірив:
Старший викладач
Орленко Сергій Петрович

Виконав:
студент гр. ІК-33
Ткачук Олексій

Київ – 2026

Мета роботи: дослідити способи збереження даних (база даних, файлова система, тощо) та отримати практичні навички щодо використання сховищ даних.

Завдання Варіант 7

Написати програму під платформу Андроїд, яка доповнює програму, що розроблена за лабораторною роботою 2, роботою зі сховищами. Тобто при натисканні на кнопку «ОК» додатково:

- здійснюється запис результату взаємодії з інтерфейсом до сховища (файл або базу даних);
- користувач інформується відповідним повідомленням щодо успішності запису.

Також інтерфейс необхідно доповнити кнопкою «Відкрити», натискання на яку призводить до переходу на іншу Діяльність, у якій відображається вміст даних, що зберігаються у сховищі. Якщо дані відсутні (сховище пусте) відобразити відповідне повідомлення. За бажанням можна додатково реалізувати оновлення та видалення даних зі сховища.

Хід роботи

1. Створення класу для роботи з базою даних

Створено новий клас DatabaseHelper. Цей клас відповідатиме за створення БД та таблиці, а також за запис і читання даних.

```
package com.example.laba3



import android.content.ContentValues

import android.content.Context
```

```
import android.database.sqlite.SQLiteDatabase

import android.database.sqlite.SQLiteOpenHelper



class DatabaseHelper(context: Context) : SQLiteOpenHelper(context,
"PizzaOrders.db", null, 1) {



    // Створюємо таблицю з двома колонками: id та текст замовлення

    override fun onCreate(db: SQLiteDatabase) {

        val createTable = "CREATE TABLE orders (id INTEGER PRIMARY KEY
AUTOINCREMENT, order_details TEXT)"

        db.execSQL(createTable)

    }



    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion:
Int) {

        db.execSQL("DROP TABLE IF EXISTS orders")

        onCreate(db)

    }



    // Функція для збереження замовлення

    fun insertOrder(orderDetails: String): Boolean {

        val db = this.writableDatabase

        val values = ContentValues()

        values.put("order_details", orderDetails)

        val result = db.insert("orders", null, values)
```

```
        return result != -1L // Повертає true, якщо успішно
    }

    // Функція для отримання всіх замовлень
    fun getAllOrders(): String {
        val db = this.readableDatabase
        val cursor = db.rawQuery("SELECT * FROM orders", null)
        var allOrders = ""

        if (cursor.moveToFirst()) {
            do {
                // Беремо текст з другої колонки (індекс 1)
                allOrders += cursor.getString(1) + "\n\n---\n\n"
            } while (cursor.moveToNext())
        }
        cursor.close()
        return allOrders
    }
}
```

2. Оновлюємо інтерфейс fragment_input.xml

Було додано нову кнопку «Відкрити», щоб переходити на екран зі збереженими даними.

```
<Button

   android:id="@+id/btnOpen"

   android:layout_width="match_parent"

   android:layout_height="wrap_content"

   android:text="Відкрити сховище"

   android:layout_marginTop="10dp" />
```

3. Оновлення логіки InputFragment.kt

Нові компоненти було об’єднано в систему. Коли натискається «ОК», дані відправляються не тільки у ViewModel , а й у базу даних, і з’являється повідомлення про успіх. Кнопка «Відкрити» має запускати нову Activity.

```
package com.example.laba3



import android.os.Bundle

import android.view.View

import android.widget.*

import androidx.fragment.app.Fragment

import androidx.fragment.app.activityViewModels



class InputFragment : Fragment(R.layout.fragment_input) {



   private val viewModel: PizzaViewModel by activityViewModels()
```

```
   override fun onViewCreated(view: View, savedInstanceState: Bundle?) {

       super.onViewCreated(view, savedInstanceState)



       val etDetails = view.findViewById<EditText>(R.id.etDetails)

       val btnOk = view.findViewById<Button>(R.id.btnOk)



       val cbMargarita = view.findViewById<CheckBox>(R.id.cbMargarita)

       val cbPepperoni = view.findViewById<CheckBox>(R.id.cbPepperoni)

       val cbCheese = view.findViewById<CheckBox>(R.id.cbCheese)



       val cbSmall = view.findViewById<CheckBox>(R.id.cbSmall)

       val cbMedium = view.findViewById<CheckBox>(R.id.cbMedium)

       val cbLarge = view.findViewById<CheckBox>(R.id.cbLarge)



       btnOk.setOnClickListener {

           val details = etDetails.text.toString()



                           val  isPizzaSelected  =  cbMargarita.isChecked  ||
cbPepperoni.isChecked || cbCheese.isChecked

            val isSizeSelected = cbSmall.isChecked || cbMedium.isChecked ||
cbLarge.isChecked



           if (details.isEmpty() || !isPizzaSelected || !isSizeSelected) {

                Toast.makeText(context, "Будь ласка, завершіть введення всіх
даних!", Toast.LENGTH_SHORT).show()
```

```
            } else {



                var orderSummary = "Замовник: $details\nОбрано піци: "

                if (cbMargarita.isChecked) orderSummary += "Маргарита "

                if (cbPepperoni.isChecked) orderSummary += "Пепероні "

                if (cbCheese.isChecked) orderSummary += "4 Сири "



                orderSummary += "\nРозміри: "

                if (cbSmall.isChecked) orderSummary += "25см "

                if (cbMedium.isChecked) orderSummary += "30см "

                if (cbLarge.isChecked) orderSummary += "40см "



                val dbHelper = DatabaseHelper(requireContext())

                val isInserted = dbHelper.insertOrder(orderSummary)



                if (isInserted) {

                      Toast.makeText(context, "Замовлення успішно збережено в
БД!", Toast.LENGTH_SHORT).show()

                } else {

                              Toast.makeText(context,  "Помилка  збереження",
Toast.LENGTH_SHORT).show()

                }



                viewModel.orderSummary.value = orderSummary
```

```
                parentFragmentManager.beginTransaction()

                    .replace(R.id.fragment_container, ResultFragment())

                    .addToBackStack(null)

                    .commit()

            }

        }



        val btnOpen = view.findViewById<Button>(R.id.btnOpen)

        btnOpen.setOnClickListener {

                    val  intent  =  android.content.Intent(requireActivity(),
StorageActivity::class.java)

            startActivity(intent)

        }

    }

}
```

4. Створення StorageActivity

Створюємо файл activity_storage.xml у папці layout

```
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:padding="16dp">



    <TextView
```

```
        android:id="@+id/tvStorageData"

        android:layout_width="match_parent"

        android:layout_height="wrap_content"

        android:textSize="16sp" />

</ScrollView>
```

Створюємо файл StorageActivity.kt

```
package com.example.laba3



import android.os.Bundle

import android.widget.TextView

import androidx.appcompat.app.AppCompatActivity



class StorageActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_storage)



        val tvStorageData = findViewById<TextView>(R.id.tvStorageData)

        val dbHelper = DatabaseHelper(this)



        val data = dbHelper.getAllOrders()



        // Перевіряємо, чи пуста база
```

```
        if (data.isEmpty()) {

            tvStorageData.text = "Сховище пусте. Зробіть перше замовлення!"

        } else {

            tvStorageData.text = "Збережені замовлення:\n\n$data"

        }

    }

}
```

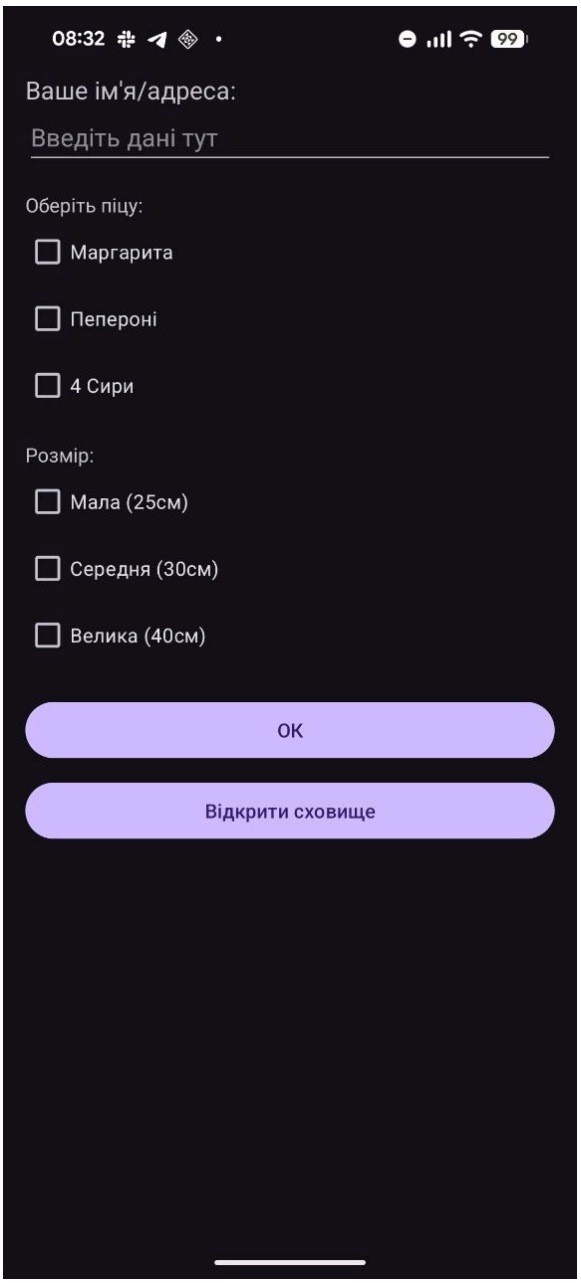
Також оновлюємо AndroidManifest.xml

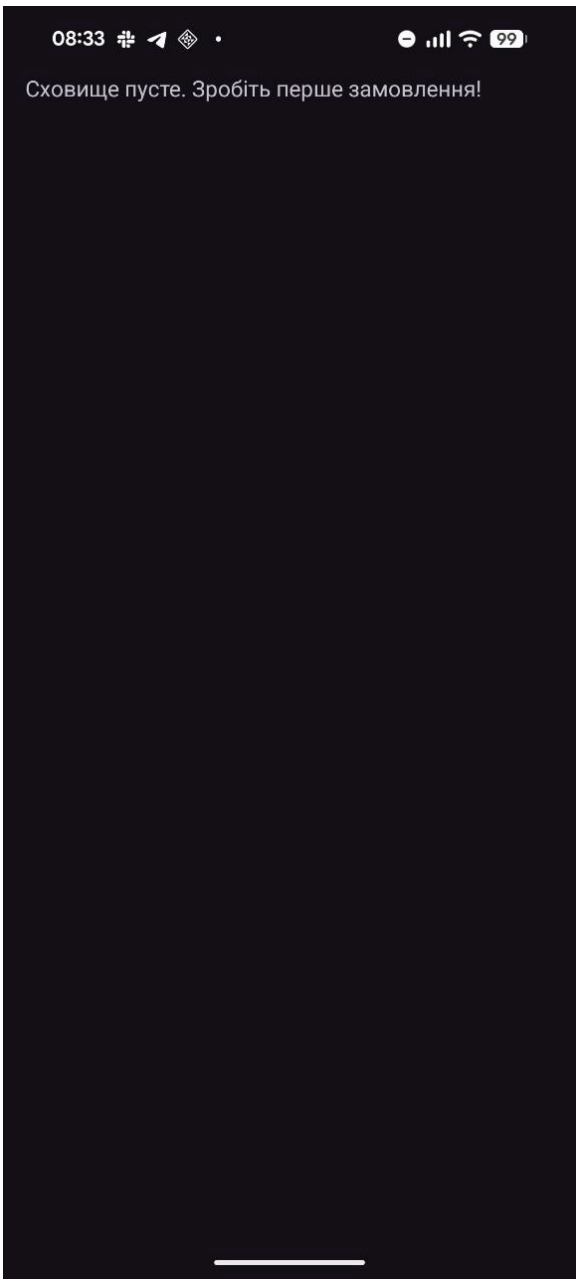
```
<?xml version="1.0" encoding="utf-8"?>

<manifest xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:tools="http://schemas.android.com/tools">



    <application

        android:allowBackup="true"

        android:dataExtractionRules="@xml/data_extraction_rules"

        android:fullBackupContent="@xml/backup_rules"

        android:icon="@mipmap/ic_launcher"

        android:label="@string/app_name"

        android:roundIcon="@mipmap/ic_launcher_round"

        android:supportsRtl="true"

        android:theme="@style/Theme.Laba2">

        <activity android:name=".StorageActivity" />

        <activity
```

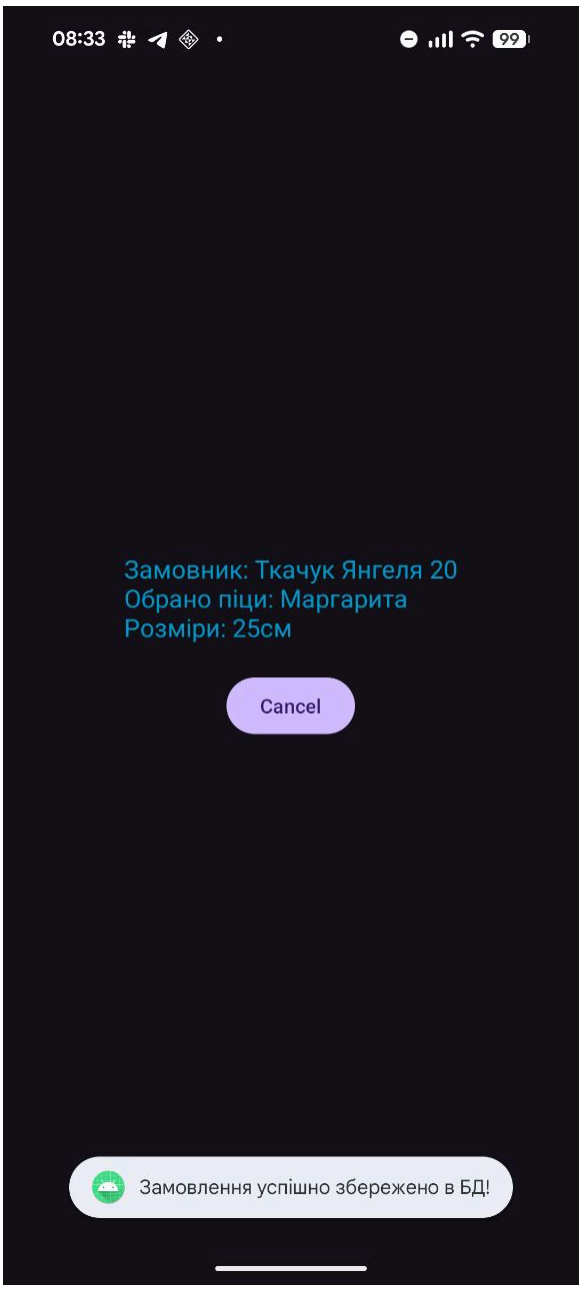
```
            android:name=".MainActivity"

            android:exported="true">

            <intent-filter>

                <action android:name="android.intent.action.MAIN" />



                <category android:name="android.intent.category.LAUNCHER" />

            </intent-filter>

        </activity>

    </application>



</manifest>
```

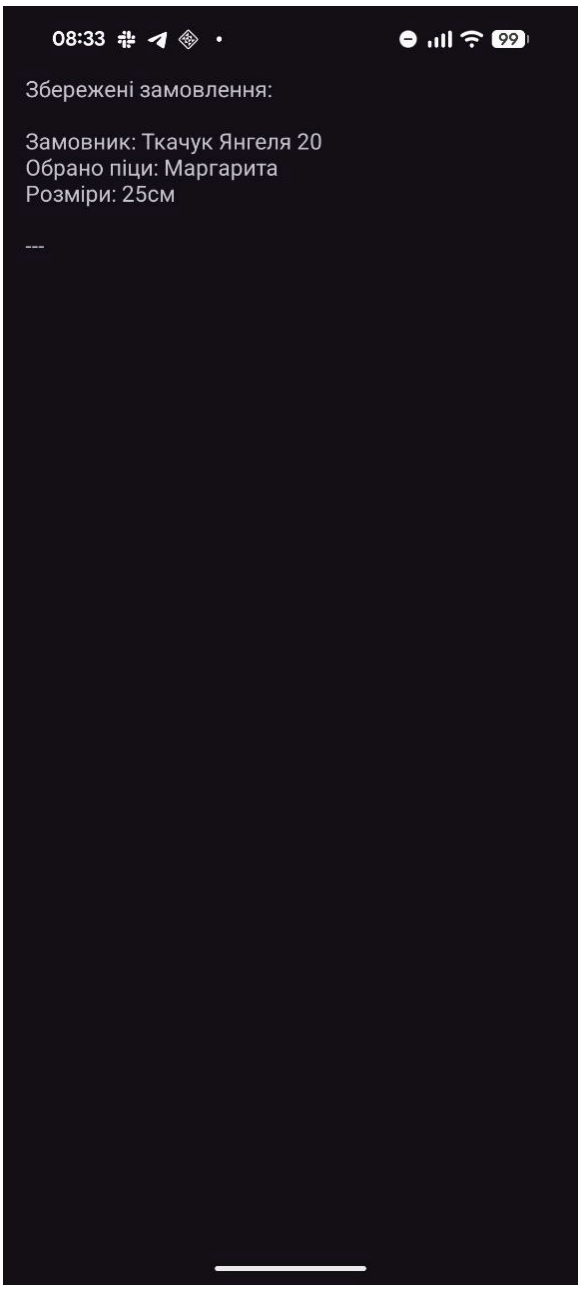
Результат роботи

Застосунок успішно доповнено функціоналом збереження даних у локальну базу даних SQLite. При натисканні кнопки «ОК» в інтерфейсі введення, окрім передачі даних, здійснюється запис сформованого замовлення до бази даних. Користувач інформується про успішність операції збереження за допомогою спливаючого повідомлення (Toast). Також інтерфейс доповнено кнопкою «Відкрити», натискання на яку ініціює перехід на нову створену Діяльність (StorageActivity). У цій Activity реалізовано зчитування даних зі сховища та їх відображення на екрані. У випадку відсутності збережених замовлень передбачено обробку пустого сховища з виведенням відповідного текстового повідомлення.


08:32
Ваше ім'я/адреса:
Введіть дані тут
Оберіть піцу:
Маргарита
Пепероні
4 Сири
Розмір:
Мала (25см)
Середня (30см)
Велика (40см)
OK
Відкрити сховище


08:33
Сховище пусте. Зробіть перше замовлення!





Висновок:

Під час виконання лабораторної роботи було реалізовано збереження даних у мобільних застосунках під Android. Програму було доповнено логікою запису результатів взаємодії з інтерфейсом до бази даних та інформування користувача про статус збереження. Крім того, було розроблено нову Activity для відображення вмісту сховища та реалізовано перевірку наявності даних із відображенням повідомлення у разі пустої бази.

Контрольні запитання:

1. **Організація роботи з налаштуваннями (SharedPreferences):** Це механізм збереження невеликих обсягів примітивних даних (рядки, числа, булеві значення) у вигляді пар «ключ-значення». Дані фізично зберігаються в XML-файлах у внутрішній пам'яті додатка і використовуються для збереження користувацьких налаштувань чи станів екрана.
2. **Типи сховищ файлів:** Існує внутрішнє сховище (Internal Storage) для приватних файлів додатка, які видаляються разом з ним, та зовнішнє (External Storage) для загальнодоступних файлів (фото, завантаження) або об'ємних даних. Їх використовують залежно від вимог до приватності та розміру файлів.
3. **Процес роботи з файлами:** Здійснюється через стандартні класи вводу/виводу (FileInputStream, FileOutputStream, File). Процес включає перевірку дозволів (для зовнішнього сховища), відкриття потоку, читання або запис байтів чи символів, та обов'язкове закриття потоку для звільнення ресурсів.
4. **Робота з SQLite через SQLiteOpenHelper:** Процес полягає у створенні класу-наслідника SQLiteOpenHelper, який бере на себе управління життєвим циклом БД через перевизначення методів onCreate() (створення таблиць) та onUpgrade() (оновлення схеми). *Переваги:* вбудованість, відсутність сервера, підтримка стандартного SQL. *Недоліки:* велика кількість шаблонного коду (бойлерплейту) та відсутність перевірки SQL-запитів на етапі компіляції.
   +1
5. **Робота з бібліотекою Room:** Room є абстракцією над SQLite. Робота будується на трьох компонентах: Entity (опис таблиць як класів), DAO (інтерфейси з анотованими SQL-запитами) та Database

(головний клас доступу). *Переваги:* перевірка запитів при компіляції, значне зменшення шаблонного коду, легка інтеграція з архітектурними компонентами.

6. **Характеристики екранів:** До основних належать фізичний розмір (діагональ у дюймах), роздільна здатність (кількість пікселів по ширині та висоті), щільність пікселів (ppi/dpi) та співвідношення сторін екрана.
7. **Класифікація технологій екранів:** Основні типи — це рідкокристалічні дисплеї (LCD, наприклад IPS, що потребують загального підсвічування) та дисплеї на органічних світлодіодах (OLED/AMOLED, де кожен піксель є самостійним джерелом світла).
8. **Поняття та характеристика сенсорних екранів:** Це пристрої вводу-виводу інформації, що реагують на дотик користувача до поверхні дисплея. Характеризуються чутливістю, часом відгуку та підтримкою технології мультитач (розпізнавання кількох дотиків одночасно).
9. **Класифікація сенсорних екранів:** Найпоширеніші типи — резистивні (реагують на фізичний тиск, застаріла технологія) та ємнісні (реагують на зміну електричної ємності при торканні пальцем, використовуються у сучасних смартфонах).
10. **Рекомендації щодо розробки інтерфейсів:** Елементи керування повинні мати достатній розмір для зручного натискання (мінімум 48x48 dp), між кнопками мають бути безпечні відступи для уникнення хибних спрацьовувань, важливі елементи слід розміщувати в зоні комфортної досяжності великого пальця, а на кожну дію має бути візуальний відгук.